

Apostila Prática: Go (Golang)

Guia Conceitual e Prático Completo — Módulo Fundação (27 Tópicos)

• Tipagem Forte & Estática | • Performance & Simplicidade | • Compilação Nativa

Parte 1: Fundamentos da Linguagem e Tipos Básicos

01 Entendendo a primeira linha

Todo código em Go pertence obrigatoriamente a um pacote (package). O compilador Go identifica o ponto de entrada de um binário executável através do pacote especial `main` e da função `main()` sem parâmetros e sem retorno.

```
package main

import "fmt"

func main() {
    fmt.Println("Olá, mundo!")
}
```

02 Declaração e atribuição

Go é fortemente e estaticamente tipada. Variáveis podem ser declaradas explicitamente com `var` ou usando inferência rápida com o operador de declaração curta `:=` (válido apenas dentro de funções).

```
var nome string = "Go"
var idade = 15          // inferência de tipo int
ativo := true           // declaração e inferência rápida
```

03 Criação de tipos

É possível criar novos tipos baseados em tipos primitivos para aumentar a segurança de tipos e expressividade do domínio, além de permitir associar métodos a eles posteriormente.

```
type ID int
type Saldo float64

var conta Saldo = 1500.50
```

04 Importando fmt e tipagem

O pacote padrão `fmt` gerencia formatação de entrada e saída. Seus principais métodos são `Println`, `Print` e `Printf` (com especificadores como `%v` para valor, `%T` para tipo e `%+v` para structs detalhadas).

```
import "fmt"

fmt.Printf("Tipo: %T | Valor: %v\n", 42, 42)
```

05 Percorrendo Arrays

Arrays em Go possuem **tamanho fixo** determinado na compilação. O tamanho faz parte da assinatura do tipo. Podem ser percorridos com o laço `for` tradicional ou usando `range`.

```
var arr [3]int = [3]int{10, 20, 30}

for idx, val := range arr {
    fmt.Printf("Posição %d: %d\n", idx, val)
}
```

Parte 2: Coleções Dinâmicas e Funções

06 Slices

Slices são estruturas dinâmicas e flexíveis construídas sobre arrays subjacentes. Possuem três componentes: ponteiro para os dados, tamanho (`len`) e capacidade (`cap`). São manipuladas com `append()` e alocadas com `make()`.

```
s := make([]int, 0, 5) // tamanho 0, capacidade 5
s = append(s, 10, 20, 30)
```

07 Maps

Maps são tabelas de dispersão (dicionários chave-valor). Para verificar se uma chave realmente existe (e não apenas retornou o *zero-value*), utiliza-se o padrão idiomático `val, ok := m[chave]`.

```
salarios := map[string]float64{
    "Ana": 5000.0,
}
if val, ok := salarios["Ana"]; ok {
    fmt.Println("Salário encontrado:", val)
}
```

08 Funções

Funções em Go suportam **múltiplos retornos** nativos (muito utilizados para retornar o par valor/erro) e retornos nomeados.

```
func dividir(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("divisão por zero")
    }
    return a / b, nil
}
```

09 Funções variádicas

Funções que aceitam uma quantidade variável de argumentos utilizando o prefixo `...T`. No corpo da função, o argumento se comporta como uma slice do tipo `T`.

```
func somar(numeros ...int) int {
    total := 0
    for _, n := range numeros { total += n }
    return total
}
```

10 Closures

Funções anônimas que mantêm referência a variáveis criadas fora de seu escopo imediato. Elas capturam e retêm o estado do ambiente léxico onde foram criadas.

```
func contador() func() int {
    i := 0
    return func() int {
        i++
        return i
    }
}
```

Parte 3: Structs, Métodos, Interfaces e Ponteiros

11 Iniciando com Structs

Structs são coleções tipadas de campos, utilizadas para modelar entidades e dados complexos sem a sobrecarga de classes tradicionais.

```
type Cliente struct {
    Nome string
    Ativo bool
}
c := Cliente{Nome: "Carlos", Ativo: true}
```

12 Composição de Structs

Go não possui herança. O reuso e extensão de comportamento são feitos por **composição (embedding)**, onde uma struct é embutida anonimamente dentro de outra.

```
type Endereco struct {
    Cidade string
}
type Usuario struct {
    Nome string
    Endereco // Composição embutida
}
```

13 Métodos em Structs

Métodos são funções declaradas com um *receiver* (receptor) vinculado a um tipo. Podem ser por valor (recebem cópia) ou por ponteiro (permitem mutação).

```
func (u Usuario) Saudacao() string {
    return "Olá, meu nome é " + u.Nome
}
```

14 Interfaces

Interfaces em Go definem contratos comportamentais. A implementação é **implícita**: se um tipo implementa todos os métodos de uma interface, ele implementa a interface automaticamente, sem palavra-chave `implements`.

```
type Notificador interface {
    Enviar(msg string) error
}
```

15 Ponteiros

Ponteiros armazenam endereços de memória. O operador & obtém o endereço de uma variável, enquanto o operador * acessa ou desreferencia o valor contido no endereço.

```
x := 10
p := &x // p guarda o endereço de memória de x
*p = 20 // altera diretamente o valor de x na memória
```

16 Quando usar ponteiros

Diretrizes práticas: Use ponteiros quando precisar modificar o estado original de uma struct, ou para evitar cópias caras de estruturas muito grandes em memória. Use valores comuns quando a struct for pequena, imutável ou quando quiser isolamento absoluto contra efeitos colaterais.

17 Ponteiros e Structs

Ao definir métodos que alteram campos internos de uma struct, o *receiver* deve obrigatoriamente ser um ponteiro (*MinhaStruct).

```
func (c *Cliente) Desativar() {
    c.Ativo = false // mutação direta no objeto original
}
```

Parte 4: Tipos Avançados, Pacotes e Controle de Fluxo

18 Interfaces vazias

A interface vazia `interface{}` (ou o alias moderno `any`) não possui métodos, logo todo e qualquer tipo em Go satisfaz essa interface. É usada quando precisamos manipular dados de tipos heterogêneos ou desconhecidos em tempo de compilação.

```
var desconhecido any = "Texto"
desconhecido = 1234
```

19 Type assertion

Permite extrair o tipo concreto subjacente a partir de uma variável do tipo `any` ou `interface`, prevenindo pânico em tempo de execução via padrão `val, ok := x.(Tipo)`.

```
if str, ok := desconhecido.(string); ok {
    fmt.Println("String:", str)
}
```

20 Generics

Introduzidos no Go 1.18, permitem parametrizar tipos em funções e structs com *type constraints*, garantindo reutilização de código com total tipagem estática.

```
func PrimeiroElemento[T any](lista []T) T {
    return lista[0]
}
```

21 a 24 Pacotes e Módulos

A modularização em Go é gerida pelo ecossistema de módulos oficiais:

- **Parte 1 & Visibilidade:** Inicialização via `go mod init meu-modulo`. Identificadores que iniciam com **letra maiúscula** são públicos (exportados); minúsculas são privados do pacote.
- **Parte 2 (Organização Local):** Criação de subdiretórios com seus próprios arquivos Go e importação interna referenciando o caminho definido no `go.mod`.
- **Parte 3 & Instalando (Dependências):** Utilização de `go get github.com/...` para baixar libs, com hashes salvos em `go.sum` para garantir reprodutibilidade.

25 Laço For

Go possui apenas uma única palavra-chave para laços: `for`. Ela substitui `while`, `do-while` e laços tradicionais:

```
// Estilo tradicional: for i := 0; i < 10; i++ {}
// Estilo while:      for condicao {}
// Loop infinito:     for {}
// Iteração:          for idx, val := range colecao {}
```

26 Condicionais

Condicionais `if` em Go permitem uma instrução de inicialização de escopo curto antes da validação. O `switch` em Go não precisa de `break` explícito entre os casos.

```
if err := executar(); err != nil {  
    return err  
}
```

27 Compilando projetos

Go compila diretamente para código de máquina nativo e estático (sem dependências externas de runtime):

- `go run main.go`: Compila em memória e executa imediatamente para desenvolvimento.
- `go build -o app`: Produz o binário final executável otimizado.
- **Cross-compilation**: Gere binários para outros sistemas sem máquinas virtuais (ex: `GOOS=linux GOARCH=amd64 go build`).